University of Bremen
Department of Computer Science
Prof. Dr. h.c. Michael Beetz, PhD

Exercises for the Lecture Advanced Methods of AI

Asking, Defining, Constraining, Deriving, Generating

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Last week you *represented* the pizza world in first-order logic and in Python. Tonight a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression, a **description**, written in the Entity Query Language (EQL). The five acts of the evening are five things the robot does with one:

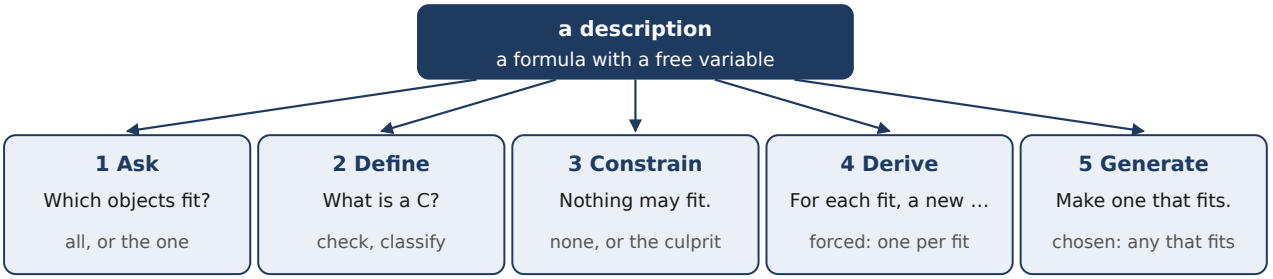


Figure 1. One description, five uses. The expression stays the same; what changes is what is done with it. Asking and constraining *read* what the robot knows; defining, deriving and generating *write* to it.

How to work

Work in pairs with one laptop. Open the notebook `03-pizza_queries.ipynb` (link on Stud.IP) and run its setup cell first. Every task says where it is done:

PAPER on this sheet: formulas, tables, drawings.

PREDICT write your expectation on this sheet *before* you run the cell. A wrong prediction is where you learn the most.

NOTEBOOK in the notebook. Every cell runs as it is, so a `TODO` you cannot solve never stops you.

PROLOGUE · 4 MINUTES

The world the robot knows

Thursday evening, 19:30

> *A small pizzeria in Bremen. A service robot works between the counter and the tables. Seven pizzas are on the counter, three tables have ordered, and two guests must not eat certain things. The robot knows all of this as objects: the classes of `pizza.owl` from last week, plus guests and orders.*

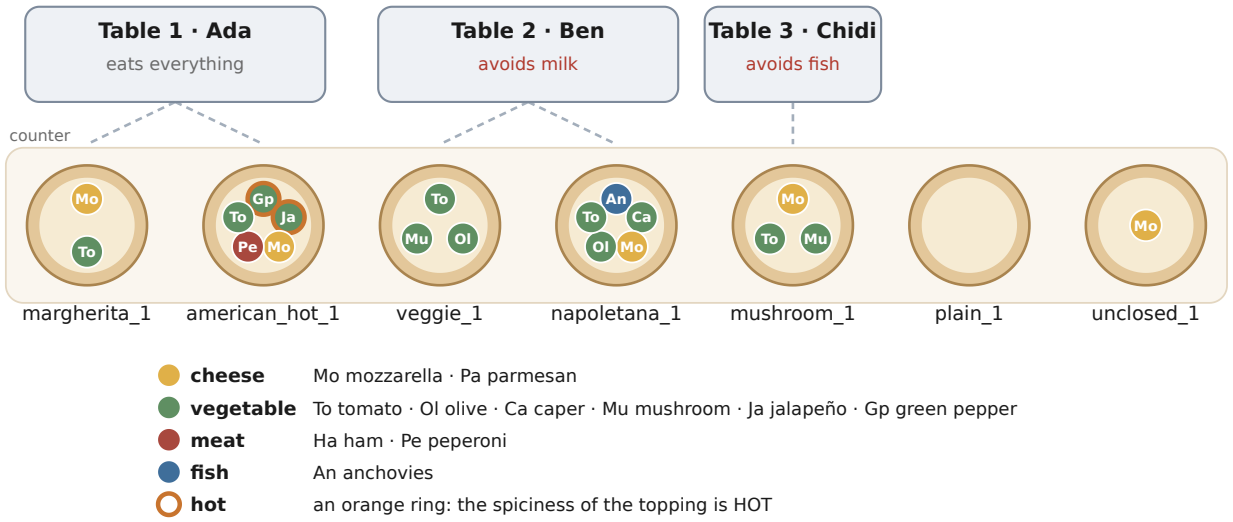


Figure 2. What the robot knows at 19:30. Lines connect each table to the pizzas it ordered. You will draw on this figure during the tutorial.

Warm-up The picture is an interpretation

PAPER

Figure 2 is an *interpretation* of first-order logic. What is its domain? What is the extension of the predicate `CheeseTopping`, and that of the relation `hasTopping`?

REFERENCE · KEEP THIS PAGE OPEN

EQL, the Entity Query Language

EQL is part of the library KRROOD. Its queries are first-order formulas written over Python objects. The central expression is a **description**: a class and a pattern of attributes.

EQL	Read it as	First-order logic
`a(Pizza)`	some object of class Pizza	a variable x with Pizza(x)
`a(Pizza)(toppings=a(CheeseTopping))`	a pizza with a cheese topping	Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t))
`a(Pizza)(toppings=[A, B])`	whose topping list is [A, B]	the whole list: exactly these, in this order
`a(PizzaTopping)(spiciness=Spiciness.HOT)`	a topping that is hot	spiciness(t) = hot
`a(Pizza)` without `.from_`	… among every pizza the robot knows	the range of x is the registry
`.from_(pizzas)`	… among these objects only	the range of x
`the(Pizza)(…)`	*the* pizza that …	exactly one object fits
`every(…)`, `no(…)` in place of `a(…)`	every, no topping is …	∀t (hasTopping → …), ¬∃t (…)
`a(PizzaTopping).from_(pizza.toppings)`	a topping of that pizza	a variable t with hasTopping(x, t)
`exists(t, φ)`, `for_all(t, φ)`, `HasType(t, C)`	some, every topping …; t is a C	∃t φ, ∀t φ; C(t)
`not_(φ)`, `or_(φ, ψ)`, `.where(φ, ψ)`	not, or, and	¬φ, φ ∨ ψ, φ ∧ ψ
`set_of(order.guest, pizza)`	return these together	the free variables of the formula
`.tolist()`	evaluate now	all assignments that satisfy it

Two things to remember

- Writing a query computes nothing: it builds a symbolic expression. Only `.tolist()` evaluates it.
- Every `a(…)` introduces a variable of its own: two `a(Pizza)` are two pizzas, as x and y are two variables. To speak about one pizza twice, name it once, `pizza = a(Pizza).from_(pizzas)`, and use `pizza`.

ACT 1 · 24 MINUTES

Asking

“Which pizzas have cheese?”

> *The chef calls across the kitchen. The cheesy pizzas go back into the oven for a minute, so that the cheese melts. The robot has to find them.*

(a) A description is a formula

PAPER Write `a(Pizza)(toppings=a(CheeseTopping))` as a first-order formula with the free variable *x*, and ring the pizzas of Figure 2 that satisfy it.

NOTEBOOK Run 1(a) and compare.

(b) Some, only, none

`a(…)` in a pattern says *some topping is* …; `every(…)` and `no(…)` take its place for *every* and *no* topping:

```
Q1 = a(Pizza)(toppings=every(VegetableTopping)).from_(pizzas)
Q2 = a(Pizza)(toppings=[a(MozzarellaTopping), a(TomatoTopping)]).from_(pizzas)
Q3 = a(Pizza)(toppings=no(CheeseTopping)).from_(pizzas)
```

PAPER Write each query as a formula.

PREDICT Mark which pizzas each query returns. Then run 1(b).

	margherita	napoletana	american hot	mushroom	veggie	plain	unclosed
Q1							
Q2							
Q3							

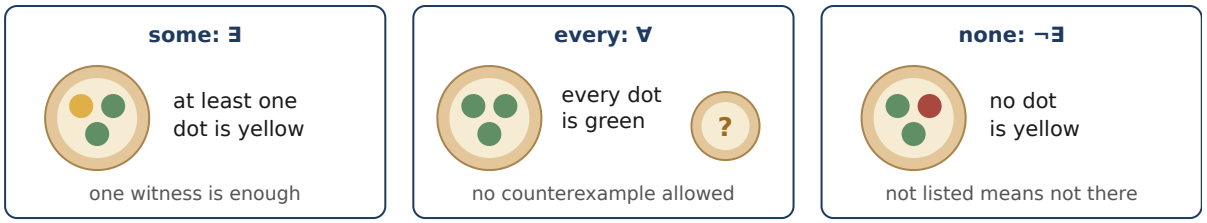


Figure 4. The three quantifier patterns, as the robot checks them on one pizza. The small pizza has no toppings at all: is every dot on it green?

> **THINK**
>
> Last week the OWL reasoner refused to classify `UnclosedPizza` (a pizza with some mozzarella) as vegetarian. In the notebook, write the query `vegetarian` (no meat and no fish topping). Is unclosed_1 an answer? Who is right, the query or the reasoner?

(c) Following the relations

> *Ada at table 1 does not like spicy food: "Is anything hot on my table?"*

Patterns nest as deep as the objects do: an order has pizzas, a pizza has toppings, a topping has a spiciness.

PAPER Write the orders that contain a pizza with a hot topping as a formula with the free variable *o*.

NOTEBOOK Write `hot_pizzas` and `hot_orders`, this time without `.from_`: the robot then searches its registry, every object of a `Symbol` class it has created. `hot_orders` may use `hot_pizzas`.

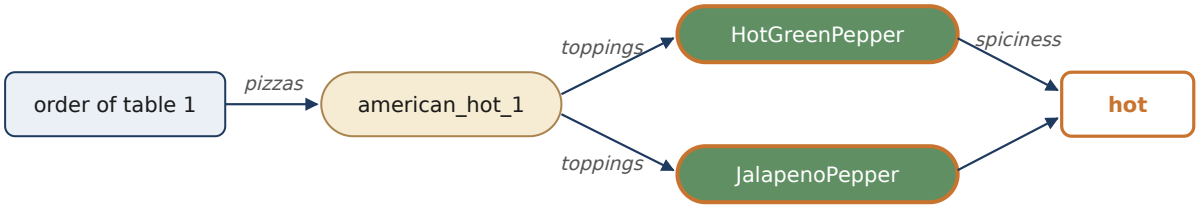


Figure 5. The paths from Ada's order to a hot topping. There are two.

PREDICT A path such as `pizza.toppings.spiciness` reads *a* topping. Does `pizza.where(not_(pizza.toppings.spiciness == Spiciness.HOT))` return the pizzas without a hot topping? _____

PREDICT The notebook then asks for the witnesses themselves: `set_of(order.guest, pizza, topping).where(topping.spiciness == Spiciness.HOT)`. How many answers does it have, while `hot_orders` has one? _____

(d) *a* or *the*?

> *Ben at table 2 waves: “Bring me the pizza with olives.”*

A command does not name an object; it describes one. `the(…)` is still a question, with one more demand: on the number of answers.

PREDICT For each command, how many pizzas fit, and what should the robot do? Then run 1(d).

0	**1**	**2 or more**
objects fit	object fits	objects fit
nothing to refer to	**refers: bring it**	**ambiguous: ask back**
`NoSolutionFound`	`the answer`	`MultipleSolutionFound`

Figure 6. `a(…)` accepts every object that fits; `the(…)` claims that exactly one fits, and raises an exception otherwise.

Command	Pizzas that fit	What the robot does
“the pizza with ham”		
“the pizza with fish”		
“the pizza with olives”		

(e) Making a description precise

The robot asks back: “The one with fish, or the one without?” Ben: “Without fish.”

PAPER Write the precise description as a formula.

NOTEBOOK Add the condition to `the(Pizza)(toppings=a(OliveTopping))` with `.where(…)`, so that `the` succeeds.

ACT 2 · 18 MINUTES

Defining

“Which pizzas may carry the green V?”

> *The chef prints tomorrow's menu, where a green V marks a vegetarian pizza, and a supplier has just delivered two frozen pizzas. In Act 1 you asked which pizzas are vegetarian. Now the word gets a meaning of its own.*

A **definition** gives a description a name. The name is then used in three ways: to *check* one candidate, to *classify* every object, and to *conclude* from the name alone.

(a) A class declaration says part of what a thing is

A Python object keeps the class it was created with: veggie_1 cannot *become* a vegetarian pizza. A label that an existing object takes on is a **role**, an object of its own that stands for the pizza and reads every attribute through to it:

```
class MenuLabel(Role[Pizza]): ...          # a menu label is a role of a pizza
class VegetarianPizza(MenuLabel): ...
class VeganPizza(VegetarianPizza): ...
```

KRROOD reads the class declarations as statements about every object of a class. Cell 2(a) prints what it reads:

Read from the declarations	First-order formula
every MozzarellaTopping is a CheeseTopping	
every Pizza has toppings, each a PizzaTopping	
every Order has a guest, which is a Guest	
every MenuLabel is a role of one Pizza	

PAPER Write each line as a formula. Can any of them ever make an object a Pizza, or a VegetarianPizza?

(b) A definition is necessary *and* sufficient

The menu's rule: *a pizza is vegetarian if and only if it has no meat topping and no fish topping.*

PAPER Write the rule as one formula. Mark the direction that is sufficient and the direction that is necessary.

PREDICT Does the robot accept frozen_1 (mozzarella, ham) and frozen_2 (tomato, capers) for the green V? Then run 2(b). _____

PREDICT The notebook then asks which pizzas are vegetarian twice: once of the registry (no `.from_`), once of `.from_(pizzas)`. Do the answers differ? _____

> **THINK**
>
> A pizza from the other branch arrives with a green V; nobody wrote down its toppings. What may the robot tell Chidi, who avoids fish? And Ben, who avoids milk?

(c) Rules with exceptions

The menu gets more labels. The chef's rule of thumb is a rule tree (Figure 7). The declaration `class VeganPizza(VegetarianPizza)` of (a) already says that every vegan pizza is vegetarian.

PREDICT Write the label of each pizza in the table, then run 2(c).

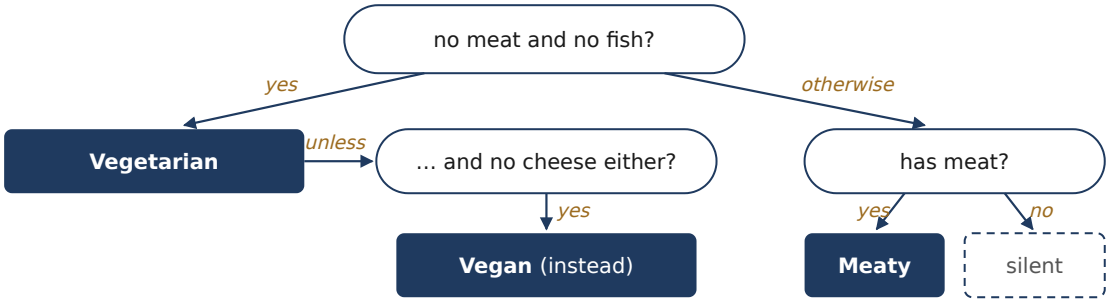


Figure 7. A rule tree: a default conclusion, an exception (*unless*, in EQL `refinement`) and an alternative (*otherwise*, in EQL `alternative`).

margherita	napoletana	american hot	mushroom	veggie	plain	unclosed

ACT 3 · 12 MINUTES

Constraining

“Ben must never get milk.”

> *Ben avoids milk. Nowhere does it say “napoletana_1 contains milk”: it has to be computed. Mozzarella is a cheese, cheese contains milk, and the pizza has that topping.*

(a) Stored or computed?

`symbolic_function` turns a Python function into a symbol of the query language, and a `Predicate` is a relation decided by a computation.

PAPER For each fact below, is it stored in the objects (S) or computed (C)?

NOTEBOOK Complete `allergens` and `IsSafeFor`.

hasTopping(*x*, *t*)	guest(*o*) = *g*	∀*t* (Mozzarella(*t*) → Cheese(*t*))	milk ∈ allergens(*x*)	IsSafeFor(*x*, *g*)

(b) A constraint is a query that must have no answer

The kitchen's rule: every pizza of an order is safe for the guest of that order.

PAPER Write the rule as a formula (hasPizza, guest, IsSafeFor), and then its negation, pushed inwards.

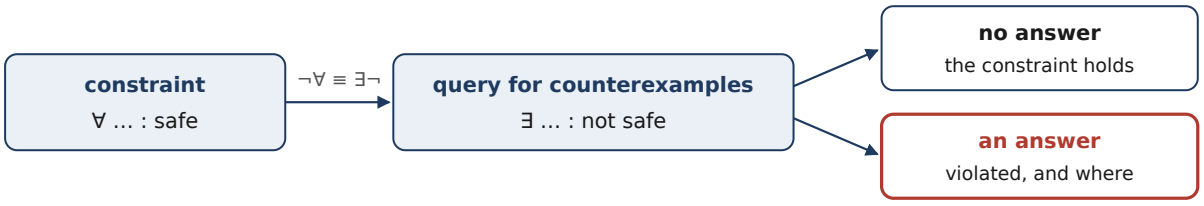


Figure 9. A universal statement is violated exactly when a counterexample exists.

PREDICT How many violations are there tonight? Draw each one as a line in Figure 2.

NOTEBOOK Write `dangerous` and check.

> **THINK**
>
> Suppose the kitchen's rule were an axiom of last week's ontology. What would the reasoner report for tonight's facts? And if the mozzarella of the Napoletana had simply not been recorded?

ACT 4 · 12 MINUTES

Deriving

The kitchen writes allergen warnings.

> *The menu must list the allergens of every pizza. Instead of computing them for every question, the kitchen writes them down once: a slip of paper on each pizza.*

The labels of Act 2 were facts about pizzas that exist. A warning is different: it is a *new* object.

(a) A rule creates objects

`inference(AllergenWarning)(pizza=D, allergen=Allergen.MILK)` creates one new warning object for every pizza that fits the description D. With D the description of Act 1(a), it is the rule

```
∀x (Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w) = milk))
```

PAPER Which part of the formula is D, and which part does the rule build?

PREDICT With a fish rule and a gluten rule for every pizza added, how many warnings are there?

NOTEBOOK Write both rules.

(b) The world moves on

> *It is busy. The cook mixes up two tickets and grates parmesan over veggie_1, Ben's pizza.*

PREDICT What do the three sources below say about veggie_1 now? Which of them protects Ben? Then run 4(b).

1. the constraint of Act 3, checked now	2. the milk warnings derived in (a)	3. the label of veggie_1 (Act 2), derived again

ACT 5 · 7 MINUTES

Generating

“A pizza with mozzarella and tomato, please.”

> *Dana sits down at table 4 and orders. Her order is a description, exactly like those of Act 1. What is new is who answers it.*

A query in EQL is a symbolic object; nothing is computed when it is written. A **backend** is whatever answers it, and the same description can be handed to different backends:

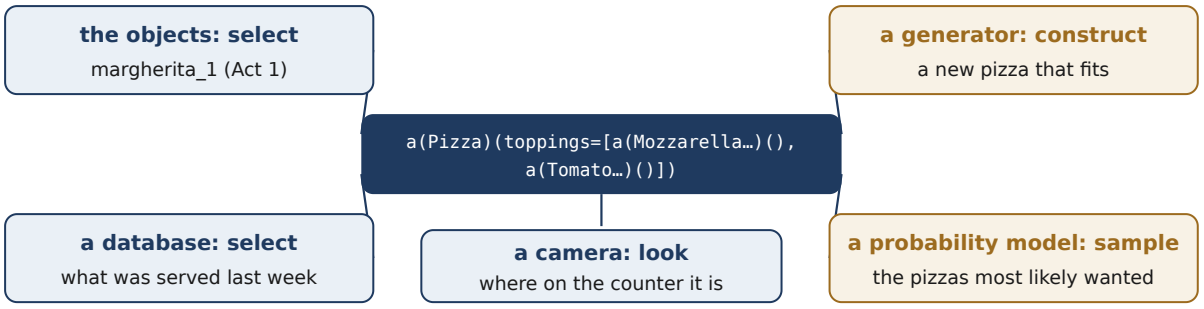


Figure 12. The description is written once, symbolically. Selecting (blue) and constructing or sampling (ochre) are different ways of answering it.

(a) Select or construct?

PREDICT What does the counter answer, and what does the kitchen answer? Dana adds “… and a green pepper, I don't mind how hot” (`spiciness=...`, left open). How many pizzas does the kitchen offer? _____

NOTEBOOK Run Act 5.

PAPER Every pizza the kitchen offered is now in the registry. Which question of tonight would get a different answer if the robot asked it again? _____

THINK

In Act 1, every answer was an object of the interpretation in Figure 2. What is an answer here? What does it mean, in terms of models, that Dana's open description has several answers?

(b) One plan, many answerers

The robot's next task is a plan with open slots, each written as a description:

> **Bring** ① a pizza that is safe for Ben, lying ② at some pose on the counter, **to** ③ the table of Ben, **holding the plate with** ④ a grasp that works. **If none is on the counter:** ⑤ have one made.

PAPER

Which kind of backend should answer each slot? Choose from **K** a query over the knowledge base, **R** rules and predicates, **L** looking with the camera, **S** sampling from a probability model, **G** a generator.

① safe pizza	② its pose	③ Ben's table	④ grasp	⑤ have one made

EPILOGUE · 5 MINUTES

The whole picture

(c) Your summary

PAPER For each use of a description, give tonight's example and what it is in logic.

Use	The question behind it	Tonight's example	In logic
1 Ask	Which objects fit?		
2 Define	What is a C?		
3 Constrain	Does anything violate it?		
4 Derive	What new object does each fit get?		
5 Generate	What could fit?		

Take home

One expression, a description, carried the whole evening. Its meaning is a formula with free variables; what you *do* with it decides whether it is a question, a definition, a constraint, the body of a rule or a specification, and which backend answers it decides whether it is checked, computed or constructed.

To think about at home: a robot tidies a kitchen. Give one description for each of the five uses, and the backend that should answer it.